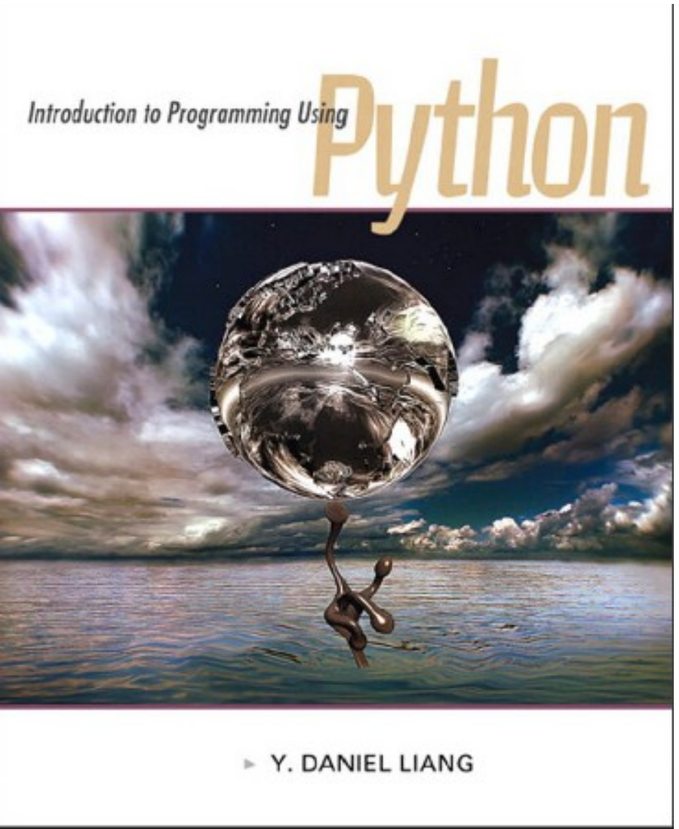


Chapter 02
Elementary Programming



Prepared By:
Eng. Eyad J. R. Elfarra

Motivations

Suppose, for example, that you need to take out a student loan. Given the loan amount, loan term, and annual interest rate, can you write a program to compute the monthly payment and total payment?

This chapter shows you how to write programs like this.

Along the way, you learn the basic steps that go into analyzing a problem, designing a solution, and implementing the solution by creating a program.

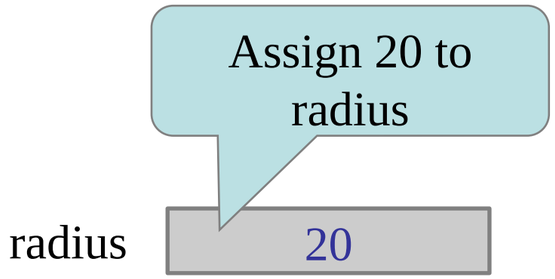
Objectives

- To write programs that perform simple computations (2.2).
- To obtain input from a program's user by using the **input** function (2.3).
- To use identifiers to name variables (2.4).
- To assign data to variables (2.5).
- To define named constants (2.6).
- To use the operators **+**, **-**, *****, ***/***, ***//***, **%**, and ****** (2.7).
- To write and evaluate numeric expressions (2.8).
- To use augmented assignment operators to simplify coding (2.9).
- To perform numeric type conversion and rounding with the **int** and **round** functions (2.10).
- To obtain the current system time by using **time.time()** (2.11).
- To describe the software development process and apply it to develop the loan payment program (2.12).
- To compute and display the distance between two points (2.13).

2.2 Simple Computation…

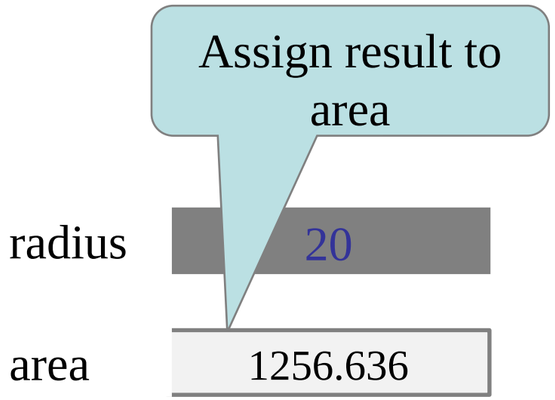
Trace a Program Execution

```
# Assign a radius
radius = 20 # radius is now 20
# Compute area
area = radius * radius * 3.14159
# Display results
print("The area for the circle of radius "
+
    str(radius) + " is " + str(area))
```



Trace a Program Execution

```
# Assign a radius
radius = 20 # radius is now 20
# Compute area
area = radius * radius * 3.14159
# Display results
print("The area for the circle of radius“,
    radius, " is "area)
```



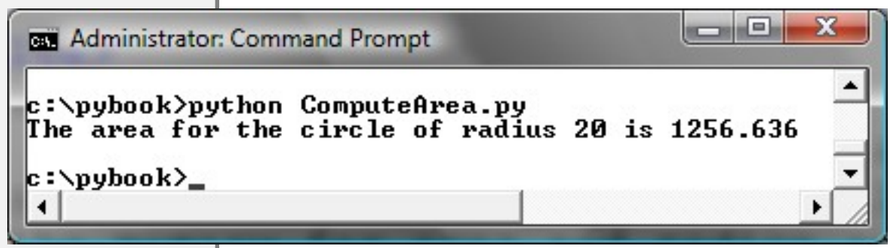
Trace a Program Execution

```
# Assign a radius
radius = 20 # radius is now 20
# Compute area
area = radius * radius * 3.14159
# Display results
print("The area for the circle of radius",
    radius, "is", area)
```

print a message to the console

radius 20

area 1256.636



2.3 User Input…

User Input

We can instruct Python to pause and read data from the user using the input() function

The input() function returns a string

```
name = input('Who are you? ')
print('Welcome', name)
```

Who are you? Chuck
Welcome Chuck

User Input

Use the eval() function to evaluate the numerical expressions.

```
variable   input('Enter a string: ')
print(‘Result: ', variable)
```

Enter a string: 51 + (54 * (3 + 2))
Result: 321

```
eval("51 + (54 * (3 + 2))")
returns 321.
```

Compute Area With Console Input

```
# Prompt the user to enter a radius
radius =eval(input("Enter a value for radius: "))

# Compute area
area = radius * radius * 3.14159

# Display results
print("The area for the circle of radius", radius,
"is", area)
```

```
Enter a value for radius: 2.5 Enter
The area for the circle of radius 2.5 is 19.6349375
```

```
Enter a value for radius: 23 Enter
The area for the circle of radius 23 is 1661.90111
```

```
# Prompt the user to enter three numbers
number1 = eval(input("Enter the first number: "))
number2 = eval(input("Enter the second number: "))
number3 = eval(input("Enter the third number: "))

# Compute average
average = (number1 + number2 + number3) / 3

# Display result
print("The average of", number1, number2, number3,
 "is", average)
```

Compute the Average

```
Enter the first number: 1 Enter
Enter the second number: 2 Enter
Enter the third number: 3 Enter
The average of 1  2  3 is 2.0
```

2.4 Identifiers…

Identifiers

Identifiers are the names that identify the elements such as variables and functions in a program.

An identifier is a sequence of characters that consists of letters, digits, underscores (_), *

An identifier cannot be a reserved word

Reserved words have special meanings in Python.

An identifier can be of any length.

Python Variable Name Rules

Must start with a letter or underscore _

Case Sensitive

Good: **spam** **eggs** **spam23** **_speed**

Bad: **23spam** **#sign** **var.12**

Different: **spam** **Spam** **SPAM**

Reserved Words / Keywords

You cannot use **reserved words** as variable names / identifiers

```
False   class   return  is      finally
None    if      for     lambda  continue
True    def     from    while   nonlocal
and     del     global  not     with
as      elif    try     or      yield
assert  else    import  pass
break   except  in      raise
```

Mnemonic Variable Names

Since we programmers are given a choice in how we choose our variable names, there is a bit of “best practice”

We name variables to help us remember what we intend to store in them (“**mnemonic**” = “memory aid”)

This can confuse beginning students because well-named variables often “sound” so good that they must be keywords

```
x1q3z9ocd = 35.0
x1q3z9afd = 12.50
x1q3p9afd = x1q3z9ocd * x1q3z9afd
print(x1q3p9afd)
```

What is this bit of code doing?

```
x1q3z9ocd = 35.0
x1q3z9afd = 12.50
x1q3p9afd = x1q3z9ocd * x1q3z9afd
print(x1q3p9afd)
```

```
a = 35.0
b = 12.50
c = a * b
print(c)
```

What are these bits of code doing?

```
x1q3z9ocd = 35.0
x1q3z9afd = 12.50
x1q3p9afd = x1q3z9ocd * x1q3z9afd
print(x1q3p9afd)
```

```
a = 35.0
b = 12.50
c = a * b
print(c)
```

What are these bits of code doing?

```
hours = 35.0
rate = 12.50
pay = hours * rate
print(pay)
```

2.5 Variables…

Variables

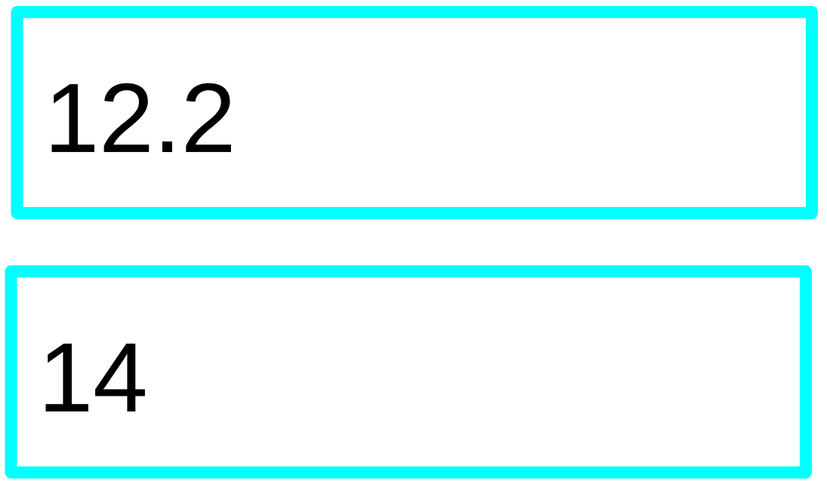
A **variable** is a named place in the memory where a programmer can store data and later retrieve the data using the **variable** “name”

Variables are used to reference values that may be changed in the program.

Programmers get to choose the names of the **variables**

You can change the contents of a **variable** in a later statement

```
x = 12.2
y = 14
```



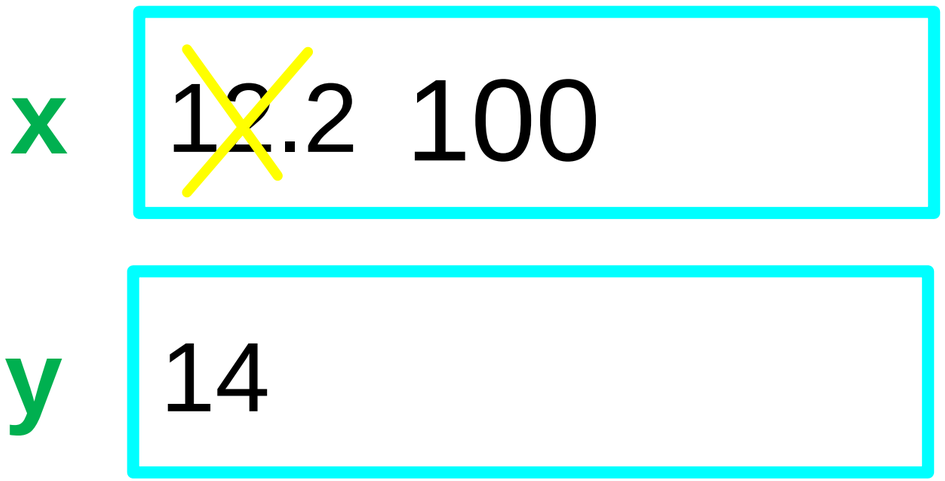
Variables

A **variable** is a named place in the memory where a programmer can store data and later retrieve the data using the **variable** “name”

Programmers get to choose the names of the **variables**

You can change the contents of a **variable** in a later statement

```
x = 12.2
y = 14
x = 100
```



Expressions…

Sentences or Lines

```
x = 2
x = x + 2
print(x)
```



Assignment statement



Assignment with expression



Print statement

Variable Operator Constant Function

Expression

```
x = 1                 # Assign 1 to variable x
radius = 1.0          # Assign 1.0 to variable radius

# Assign the value of the expression to x
x = 5 * (3 / 2) + 3 * 2

x = y + 1     # Assign the addition of y and 1 to x
area = radius * radius * 3.14159 # Compute area
```

2.6 Simultaneous Assignments…

Assignment Statements

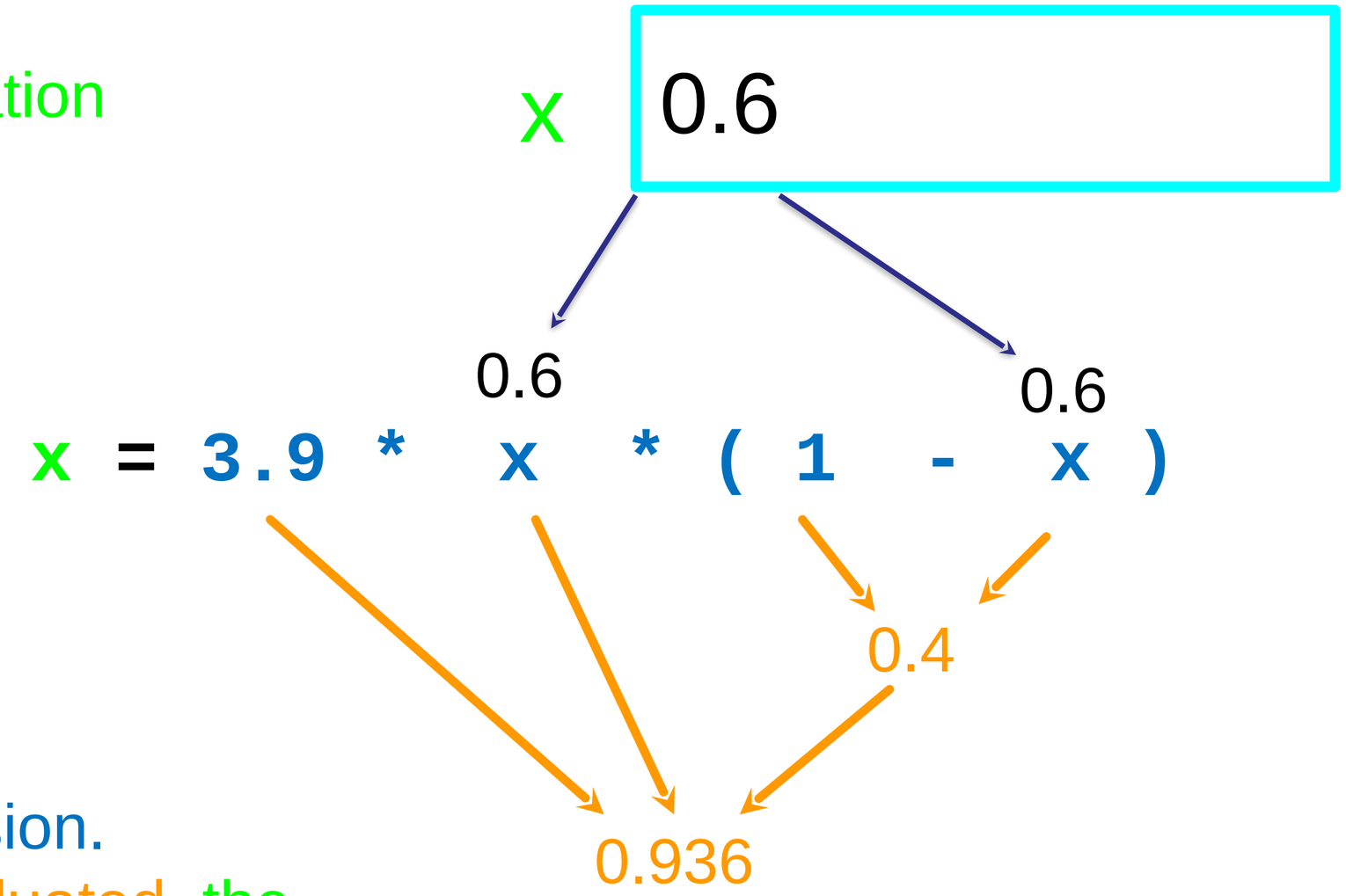
We assign a value to a variable using the assignment statement (=)

An assignment statement consists of an **expression on the right-hand side** and a **variable** to store the result

```
x = 3.9 * x * ( 1 - x )
```

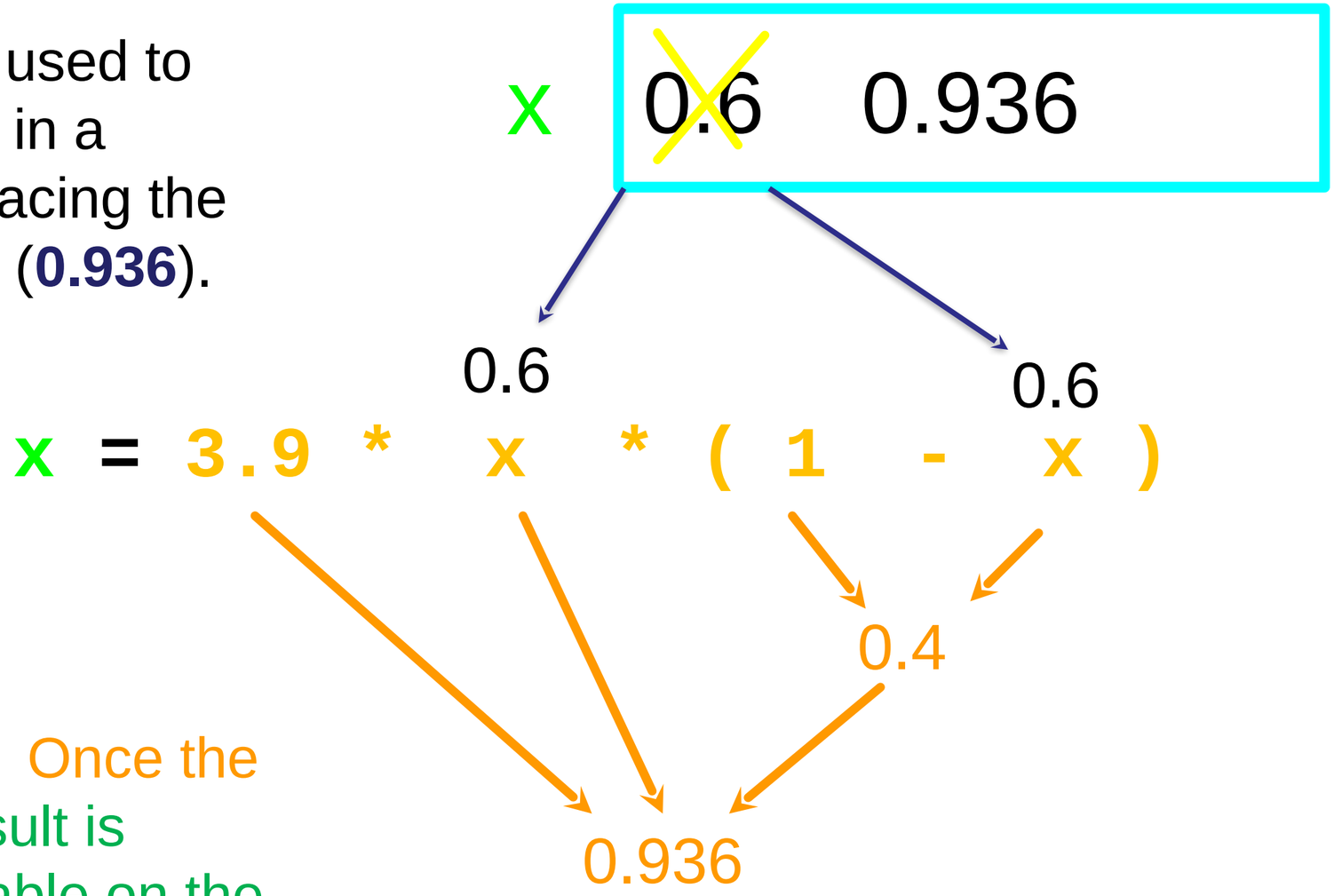
```
1 = x # Wrong
```

A variable is a memory location
used to store a value (0.6)



The right side is an expression.
Once the expression is evaluated, the
result is placed in (assigned to) x.

A variable is a memory location used to store a value. The value stored in a variable can be updated by replacing the old value (**0.6**) with a new value (**0.936**).



The right side is an expression. Once the expression is evaluated, the result is placed in (assigned to) the variable on the left side (i.e., x).

Simultaneous Assignment

It tells Python to evaluate all the expressions on the right and assign them to the corresponding variable on the left simultaneously

```
var1, var2, ..., varn = exp1,
    exp2, ..., expn
```

```
x, y = y, x # Swap x with y
```

ComputeAverageWithSimultaneousAssignment.py

```
# Prompt the user to enter three numbers
number1, number2, number3 = eval(input(
"Enter three numbers separated by commas: "))

# Compute average
average = (number1 + number2 + number3) / 3

# Display result
print("The average of", number1, number2, number3
"is", average)
```



```
Enter three numbers separated by commas: 1, 2, 3 ↵Enter
The average of 1  2  3 is 2.0
```

2.7 Constants…

Constants

- **Fixed values** such as numbers, letters, and strings, are called "**constants**" because their value does not change

  Numeric **constants** are as you expect

  String **constants** use single quotes (') or double quotes (")

```
>>> print(123)
123
>>> print(98.6)
98.6
>>> print('Hello world')
Hello world
```

Named Constants

- The value of a variable may change during the execution of a program, but a ***named constant*** or simply ***constant*** represents permanent data that never changes.
- Python does not have a special syntax for naming constants.
- You can simply create a variable to denote a constant.

Named Constants

There are three benefits of using constants:

1. You don’t have to repeatedly type the same value if it is used multiple times.
2. If you have to change the constant’s value (e.g., from 3.14 to 3.14159 for PI), you need to change it only in a single location in the source code.
3. Descriptive names make the program easy to read.
4. To Make it easy to distinguished from variable All characters in the identifier are written in capital case .

Named Constants

```
# Assign a radius
radius = 20 # radius is now 20
# Compute area
area = radius * radius * 3.14159
# Display results
print("The area for the circle of radius",
    radius, "is", area)
```

```
# Assign a radius
radius = 20 # radius is now 20
# Compute area
PI = 3.14159
area = radius * radius * PI
# Display results
print("The area for the circle of radius",
    radius, "is", area)
```

2.8 Numeric Data Types and Operators…

Numerical Data Types

- integer: e.g., 3, 4
- float: e.g., 3.0, 4.0,4.5

Numeric Expressions

Because of the **lack** of mathematical **symbols** on computer keyboards - we use “computer-speak” to express the classic math operations

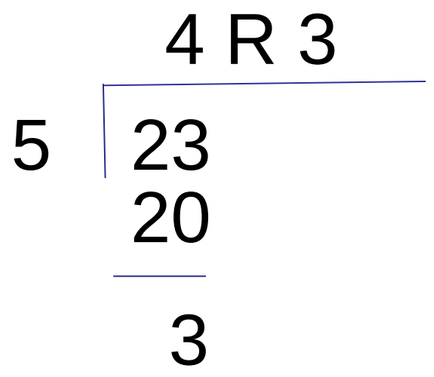
Asterisk is multiplication

Exponentiation (raise to a power) looks different than in math.

Numeric Expressions

```
>>> xx = 2
>>> xx = xx + 2
>>> print(xx)
4
>>> yy = 440 * 12
>>> print(yy)
5280
>>> zz = yy / 1000
>>> print(zz)
5.28
```

```
>>> jj = 23
>>> kk = jj % 5
>>> print(kk)
3
>>> print(4 ** 3)
64
```



Numeric Operators

Name	Meaning	Example	Result
+	Addition	34 + 1	35
-	Subtraction	34.0 – 0.1	33.9
*	Multiplication	300 * 30	9000
/	Float Division	1 / 2	0.5
//	Integer Division	1 // 2	0
**	Exponentiation	4 ** 0.5	2.0
%	Remainder	20 % 3	2

Float Division

Float division produces a floating point result

```
>>> print(10 / 2)
5.0
>>> print(9 / 2)
4.5
>>> print(99 / 100)
0.99
>>> print(10.0 / 2.0)
5.0
>>> print(99.0 / 100.0)
0.99
```

This was different in Python 2.x

Floor (Integer) Division //

- The // operator works as expected with positive numbers

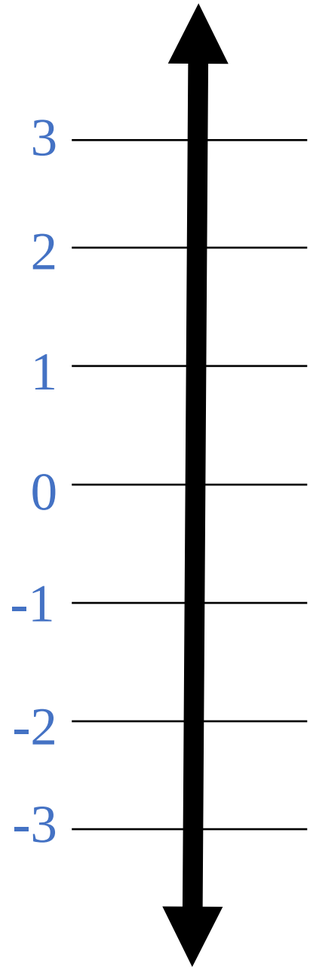
evaluates to 7 5 // 37

- 37 / 5 is 7 remainder 2 in Euclidian Division
- **https://en.wikipedia.org/wiki/Euclidean_division**
- But …

evaluates to -8 5- // 37

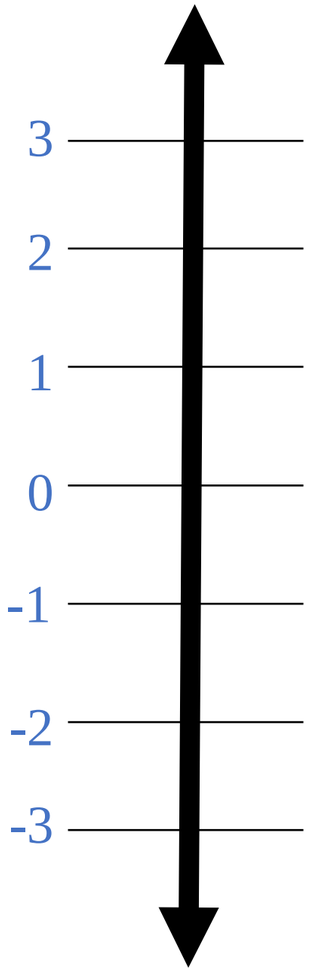
Floor Division

- From the Python Reference:
- "**Division of integers yields a float**, *while floor division of integers results in an integer*; the result is that of mathematical division with the ‘**floor**’ function applied to the result."
- Think of a vertical number line with increasing values going up



Floor and Floor Division

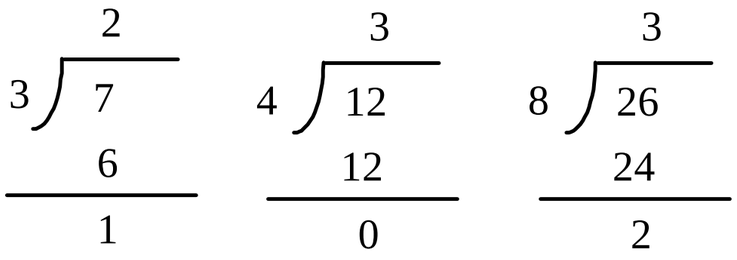
- math.floor(-1.95) evaluates to -2
- math.floor(1.95) evaluates to 1
- math.floor(1.0) evaluates to 1
- math.floor(-2.05) evaluates to -3
- math.floor(-2.55) evaluates to -3
- The floor is the integer value "underneath" the number unless it is a whole number

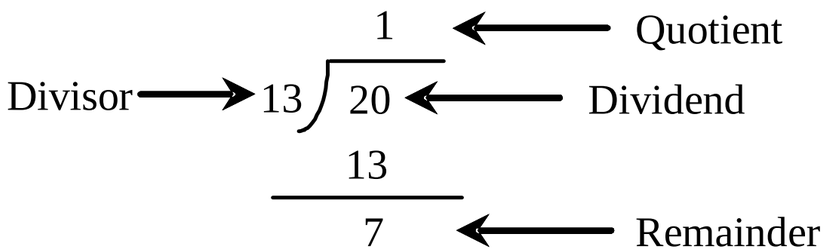


Floor Division

- So, why does **37 // -5** evaluate to **-8**?
  - **37 / -5** evaluates to **-7.4**
  - **math.floor(-7.4)** evaluates to **-8**

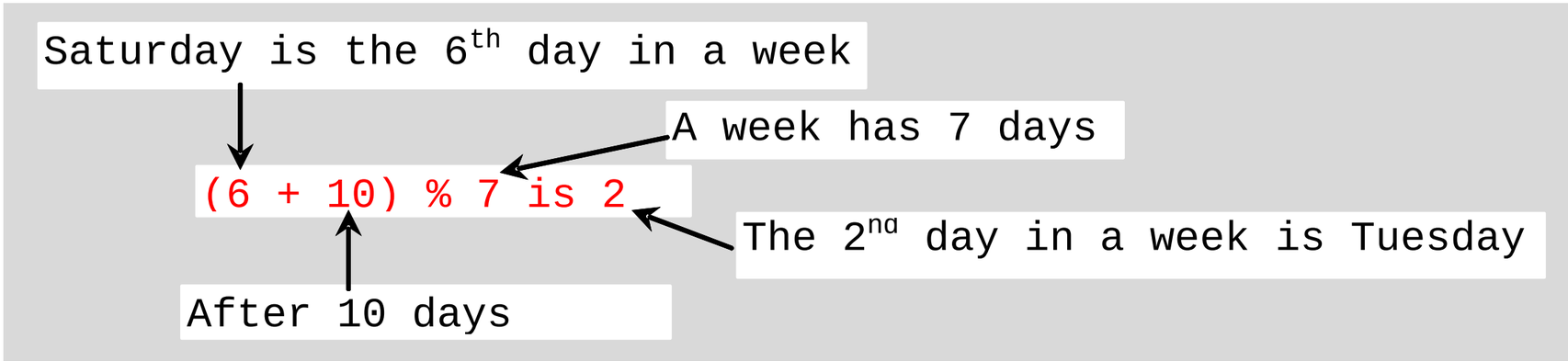
The % Operator





Modulo Operator

Remainder is very useful in programming. For example, **an even number % 2 is always 0** and an **odd number % 2 is always 1**. So you can use this property to determine whether a number is even or odd. Suppose today is Saturday and you and your friends are going to meet in 10 days. What day is in 10 days? You can find that day is Tuesday using the following expression:



More on the Modulo Operator

- Again, for **positive** values Modulo works as expected if performing Euclidian Division
- 37 % 5 evaluates to 2
- 37 / 5 is 7 remainder 2
- But with **negative** values?
- 37 % -5 evaluates to -3 even though
  37 / - 5 is -7 remainder -2
- But recall, floor division …

More on the Modulo Operator

- Recall, **37 // -5** evaluates to **-8** in Python due to floor division.
- So **-5 * -8** evaluates to **40**
- So what do we need to add to **-5 * -8** to get **37**?
- **-3** !!!! Ah-Ha!!!!
- "The floor division and modulo operators are connected by the following identity:
  x == (x // y) * y + (x % y)

Problem: Displaying Time

Write a program that obtains hours and minutes from seconds.

```
# Prompt the user for input
seconds = eval(input("Enter an integer for seconds: "))

# Get minutes and remaining seconds
minutes = seconds // 60 # Find minutes in seconds
remainingSeconds = seconds % 60 # Seconds remaining
print(seconds, "seconds is", minutes,"minutes and", remainingSeconds,
"seconds")
```

Overflow

When a variable is assigned a value that is **too large (*in size*)** to be stored, it causes ***overflow***. For example, executing the following statement causes *overflow*.

```
>>>245.0 ** 1000

OverflowError: 'Result too large'
```

Underflow

- When a ***floating-point*** number is ***too small*** (i.e., too close to zero) to be stored, it causes *underflow*.
- **Python approximates it to zero**. So normally you should not be concerned with underflow.

Scientific Notation

Floating-point literals can also be specified in **scientific notation**,

for example, **1.23456e+2**, same as **1.23456e2**, is equivalent to **123.456**,

and **1.23456e-2** is equivalent to **0.0123456**.

E (or e) represents an **exponent** and it can be either in **lowercase or uppercase**.

2.9 Evaluating Expressions and Operator Precedence

Arithmetic Expressions

$$\frac{3+4x}{5} - \frac{10(y-5)(a+b+c)}{x} + 9\left(\frac{4}{x} + \frac{9+x}{y}\right)$$

is translated to

```
(3+4*x)/5 – 10*(y-5)*(a+b+c)/x + 9*(4/x + (9+x)/y)
```

Order of Evaluation

When we string operators together - Python must know **which one to do first**

This is called “**operator precedence**”

Which operator “**takes precedence**” over the others?

```
x = 1 + 2 * 3 - 4 / 5 ** 6
```

Operator Precedence Rules

Highest precedence rule to lowest precedence rule:

Parentheses are always respected

Exponentiation (raise to a power)

Multiplication, Division, and Remainder

Addition and Subtraction

Left to right

Parenthesis

Power

Multiplication

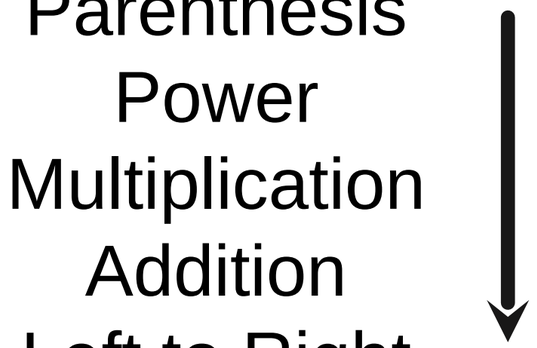
Addition

Left to Right



```
>>> x = 1 + 2 ** 3 / 4 * 5
>>> print(x)
11.0
>>>
```

Parenthesis
Power
Multiplication
Addition
Left to Right



```
1 + 2 ** 3 / 4 * 5
1 + 8 / 4 * 5
1 + 2 * 5
1 + 10
11
```

Operator Precedence

Parenthesis
Power
Multiplication
Addition
Left to Right



Remember the rules **top to bottom**

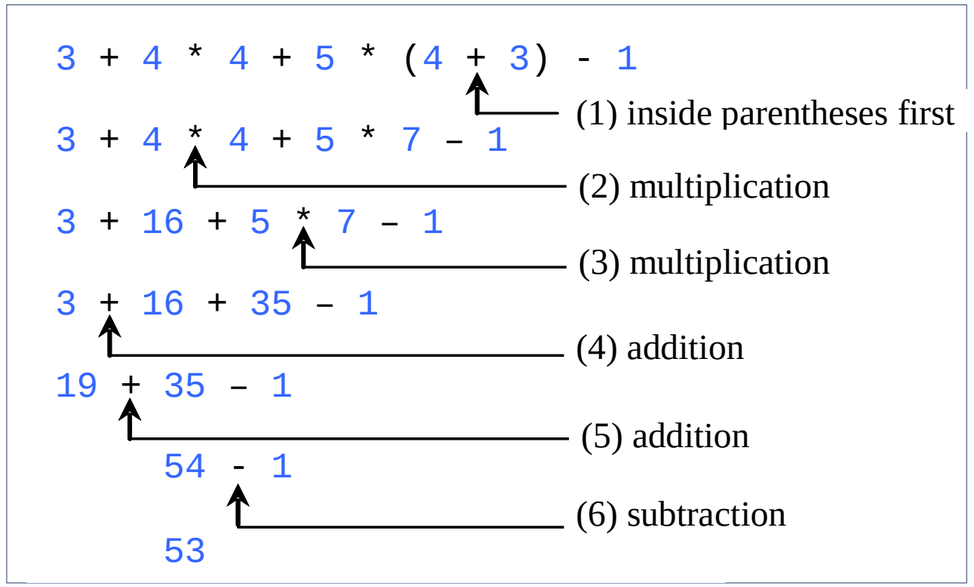
When writing code - use parentheses

When writing code - keep mathematical expressions simple enough that they are easy to understand

Break long series of mathematical operations up to make them more clear

How to Evaluate an Expression

Though Python has its own way to evaluate an expression behind the scene, the result of a Python expression and its corresponding arithmetic expression are the same. Therefore, you can safely apply the arithmetic rule for evaluating a Python expression.



2.10 Augmented Assignment Operators

Augmented Assignment Operators

Operator	*Example*	*Equivalent*
`+=`	`i += 8`	`i = i + 8`
`-=`	`f -= 8.0`	`f = f - 8.0`
`*=`	`i *= 8`	`i = i * 8`
`/=`	`i /= 8`	`i = i / 8`
`%=`	`i %= 8`	`i = i % 8`

2.11 Type Conversions and Rounding

What Does “Type” Mean?

In Python variables, literals, and constants have a “**type**”

Python knows the **difference** between an integer number and a string

For example **“+”** means “addition” if something is a number and “concatenate” if something is a string

```
>>> ddd = 1 + 4
>>> print(ddd)
5
>>> eee = 'hello ' + 'there'
>>> print(eee)
hello there
```

concatenate = put together

Types

```
data type(value)

i.e., int(4.5) => 4
      float(4) => 4.0
      str(4) => “4”

round(4.6) => 5
round(4.5) => 4
```

Type Matters

Python knows what “**type**” everything is

Some operations are prohibited

You cannot “add 1” to a string

We can ask Python what type something is by using the **type()** function

```
>>> eee = 'hello ' + 'there'
>>> eee = eee + 1
Traceback (most recent call last):
 File "<stdin>", line 1, in
<module>TypeError: Can't convert
'int' object to str implicitly
>>> type(eee)
<class'str'>
>>> type('hello')
<class'str'>
>>> type(1)
<class'int'>
>>>
```

Several Types of Numbers

Numbers have two main types

- **Integers** are whole numbers:
  -14, -2, 0, 1, 100, 401233

- **Floating Point Numbers** have decimal parts: -2.5 , 0.0, 98.6, 14.0

There are other number types - they are variations on float and integer

```
>>> xx = 1
>>> type (xx)
<class 'int'>
>>> temp = 98.6
>>> type(temp)
<class'float'>
>>> type(1)
<class 'int'>
>>> type(1.0)
<class'float'>
>>>
```

Type Conversions

When you put an integer and floating point in an expression, the integer is **implicitly** converted to a float

You can control this with the built-in functions int() and float()

```
>>> print(float(99) + 100)
199.0
>>> i = 42
>>> type(i)
<class'int'>
>>> f = float(i)
>>> print(f)
42.0
>>> type(f)
<class'float'>
>>>
```

String Conversions

You can also use **int()** and **float()** to convert between strings and integers

You will get an **error** if the string does not contain numeric characters

```
>>> sval = '123'
>>> type(sval)
<class 'str'>
>>> print(sval + 1)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError: Can't convert 'int' object
to str implicitly
>>> ival = int(sval)
>>> type(ival)
<class 'int'>
>>> print(ival + 1)
124
>>> nsv = 'hello bob'
>>> niv = int(nsv)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: invalid literal for int()
with base 10: 'x'
```

Converting User Input



If we want to read a number from the user, we must convert it from a string to a number using a type conversion function

Later we will deal with bad input data

```
inp = input('Europe floor?')
usf = int(inp) + 1
print('US floor', usf)
```

Problem: Keeping Two Digits After Decimal Points

Write a program that displays the sales tax with two digits after the decimal point.

```
# Prompt the user for input
purchaseAmount = eval(input("Enter purchase amount: "))

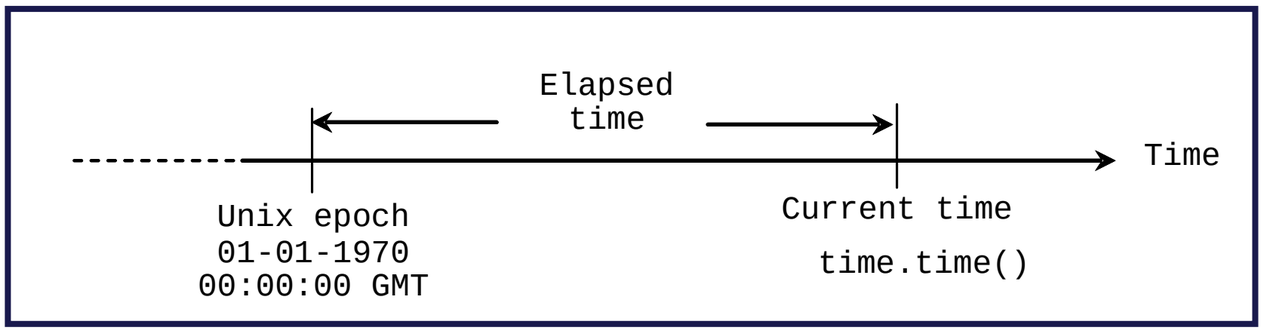
# Compute sales tax
tax = purchaseAmount * 0.06

# Display tax amount with two digits after decimal point
print("Sales tax is", int(tax * 100) / 100.0)
```

Problem: Displaying Current Time

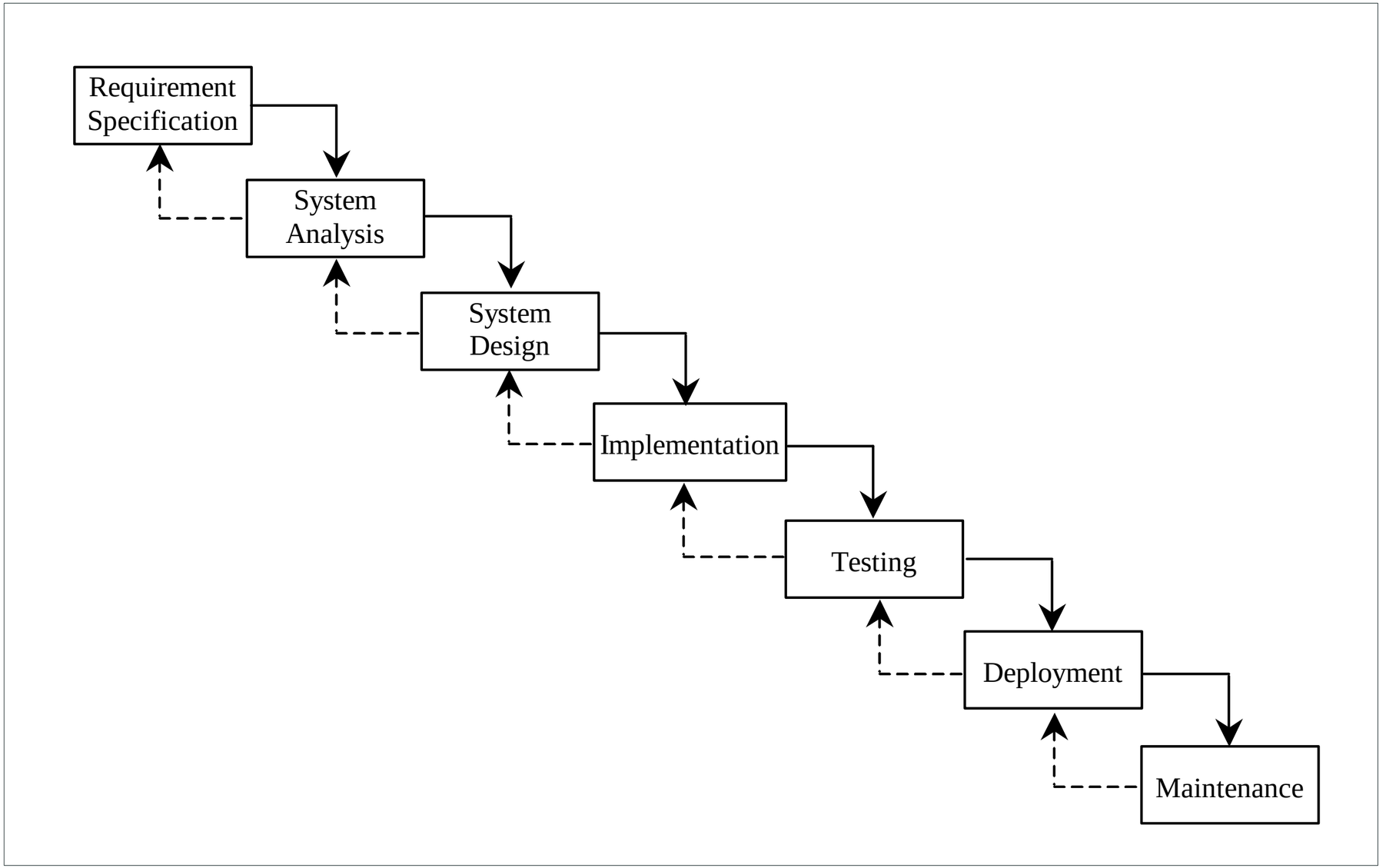
Write a program that displays current time in GMT in the format **hour:minute:second** such as 1:45:19.

The **time.time()** function returns the current time in seconds with millisecond precision since **the midnight, January 1, 1970 GMT**. (1970 was the year when the Unix operating system was formally introduced.) You can use this function to obtain the current time, and then compute the current second, minute, and hour as follows.

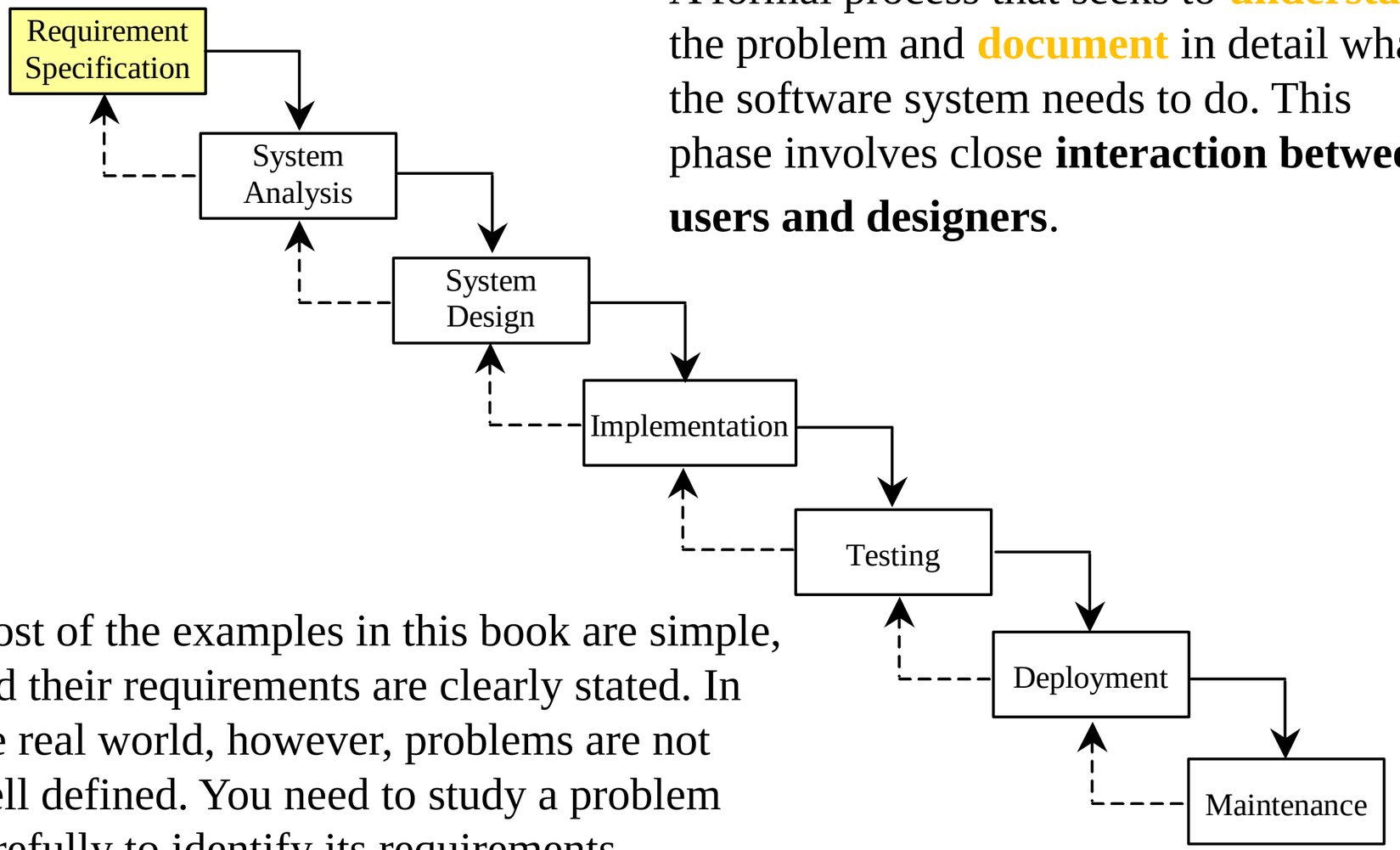


2.13 Software Development Process…

Software Development Process



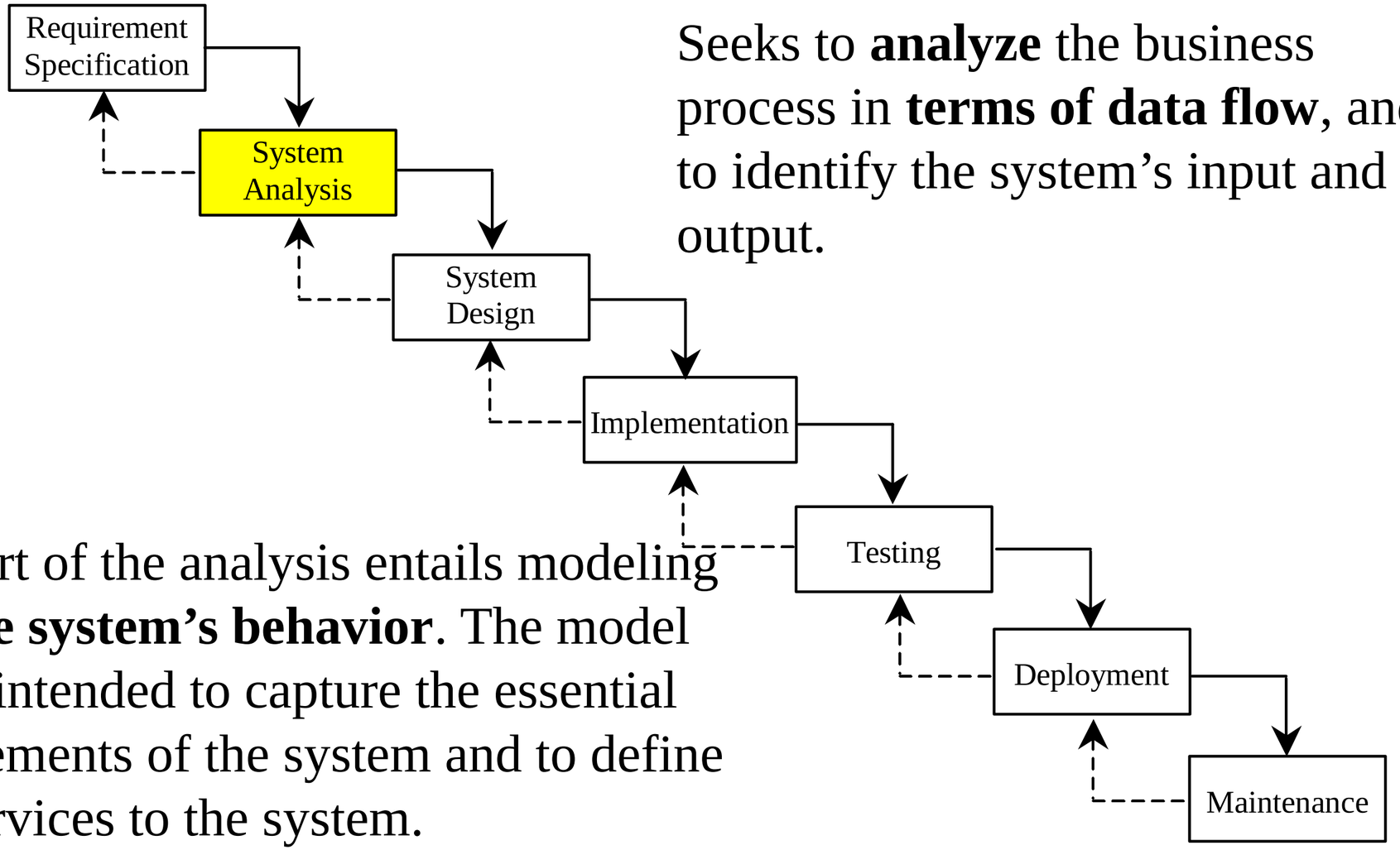
Requirement Specification



A formal process that seeks to **understand** the problem and **document** in detail what the software system needs to do. This phase involves close **interaction between users and designers**.

Most of the examples in this book are simple, and their requirements are clearly stated. In the real world, however, problems are not well defined. You need to study a problem carefully to identify its requirements.

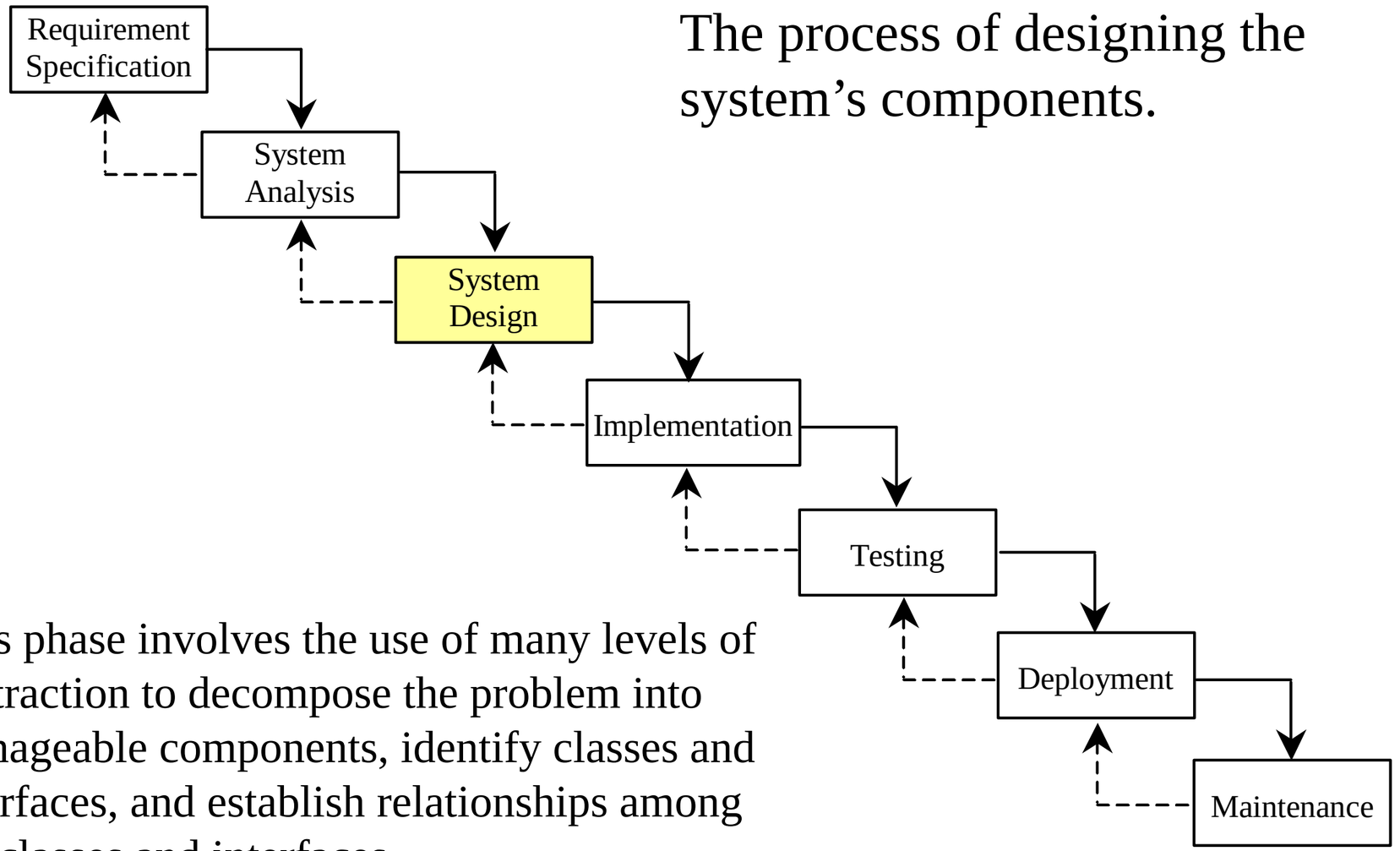
System Analysis



Seeks to **analyze** the business process in **terms of data flow**, and to identify the system's input and output.

Part of the analysis entails modeling **the system's behavior**. The model is intended to capture the essential elements of the system and to define services to the system.

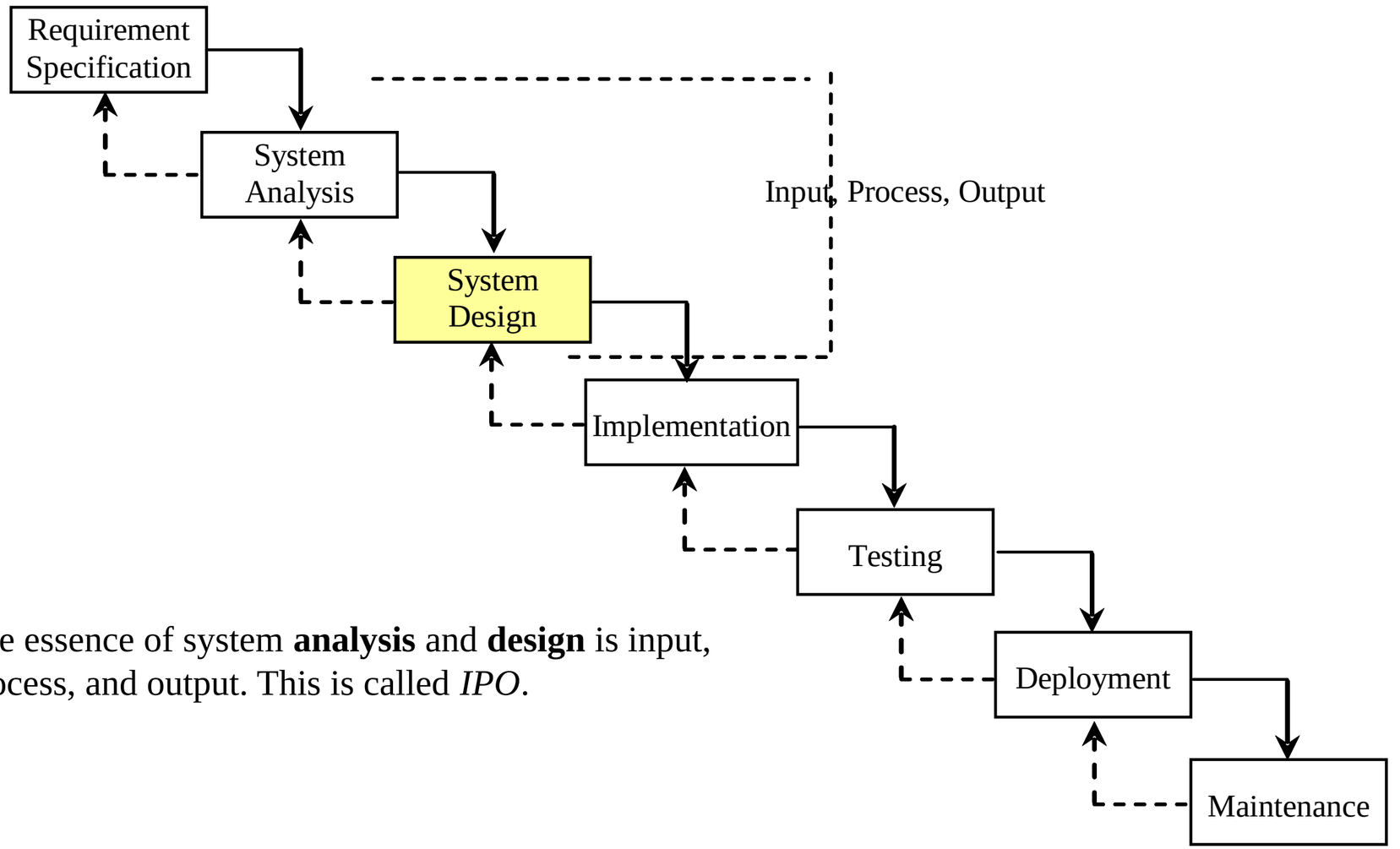
System Design



The process of designing the system's components.

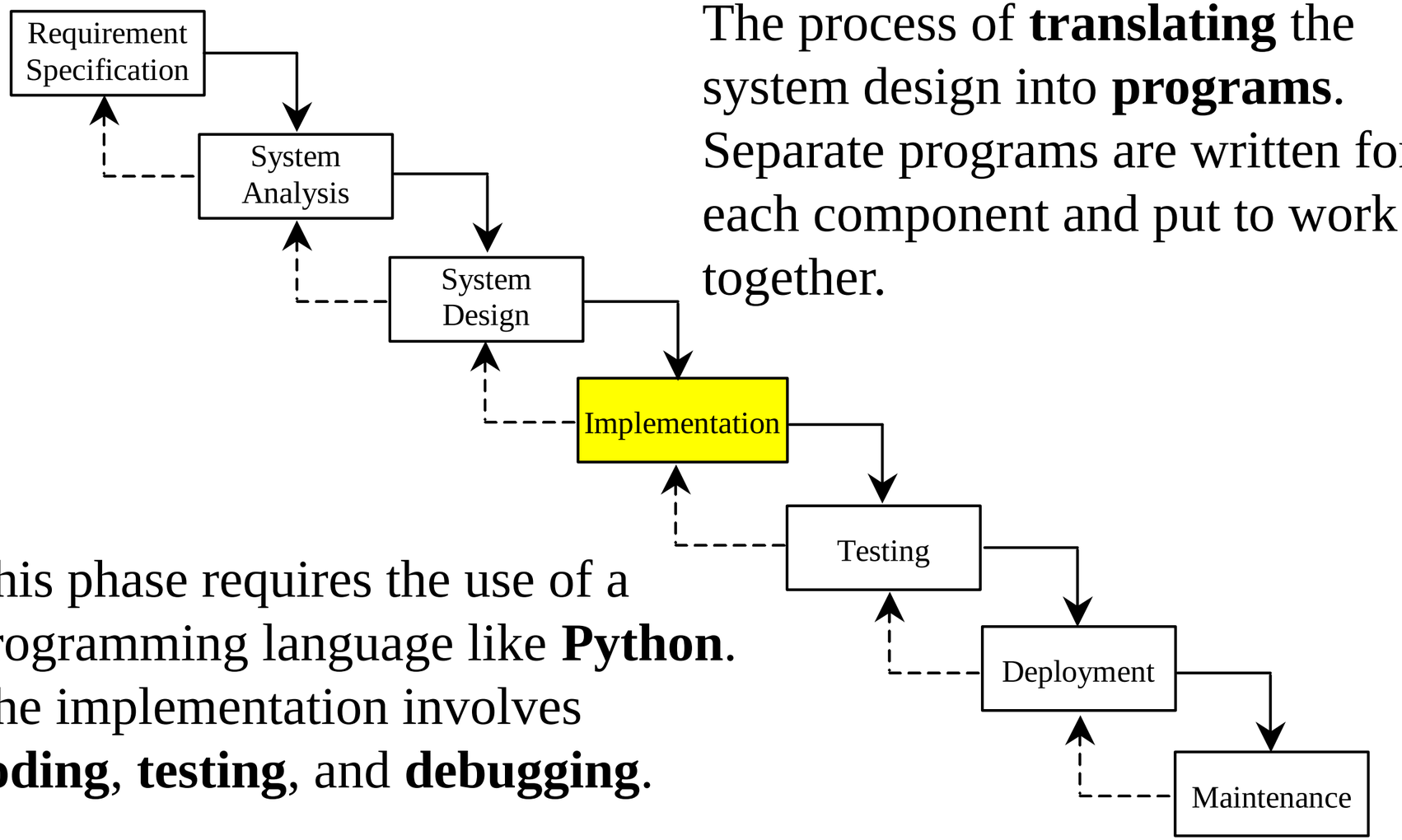
This phase involves the use of many levels of abstraction to decompose the problem into manageable components, identify classes and interfaces, and establish relationships among the classes and interfaces.

IPO



The essence of system **analysis** and **design** is input, process, and output. This is called *IPO*.

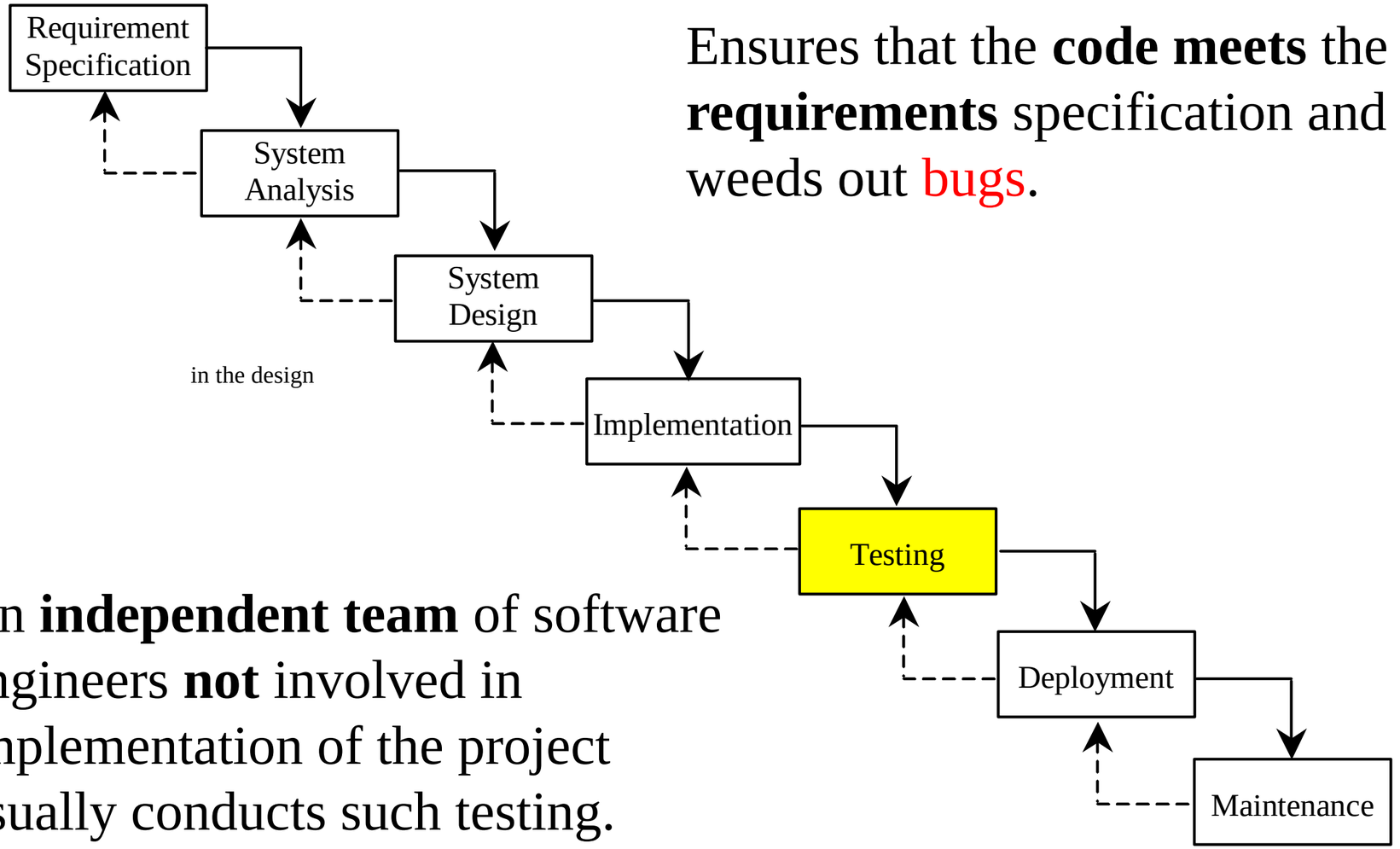
Implementation



The process of **translating** the system design into **programs**. Separate programs are written for each component and put to work together.

This phase requires the use of a programming language like **Python**. The implementation involves **coding**, **testing**, and **debugging**.

Testing

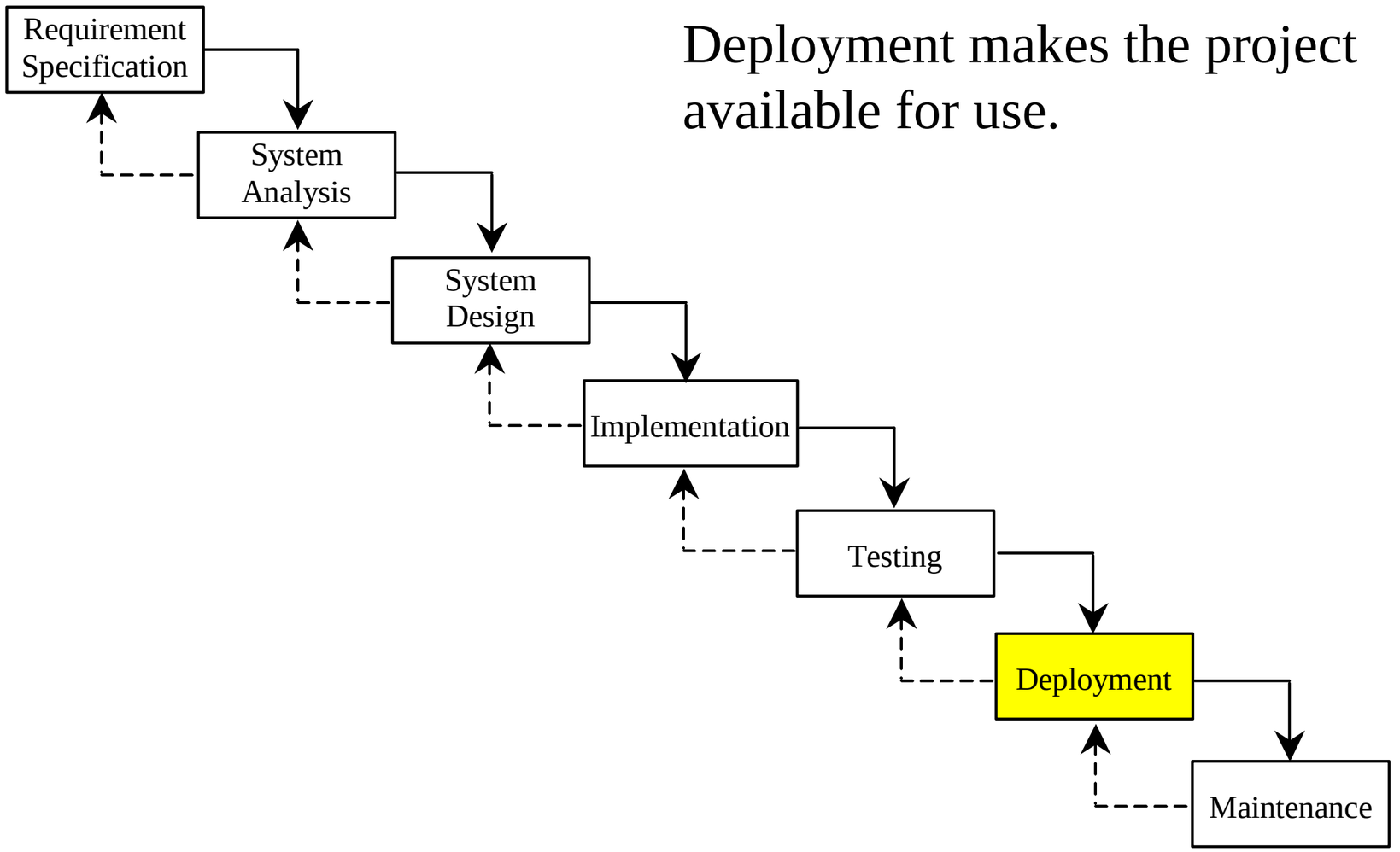


Ensures that the **code meets** the **requirements** specification and weeds out bugs.

in the design

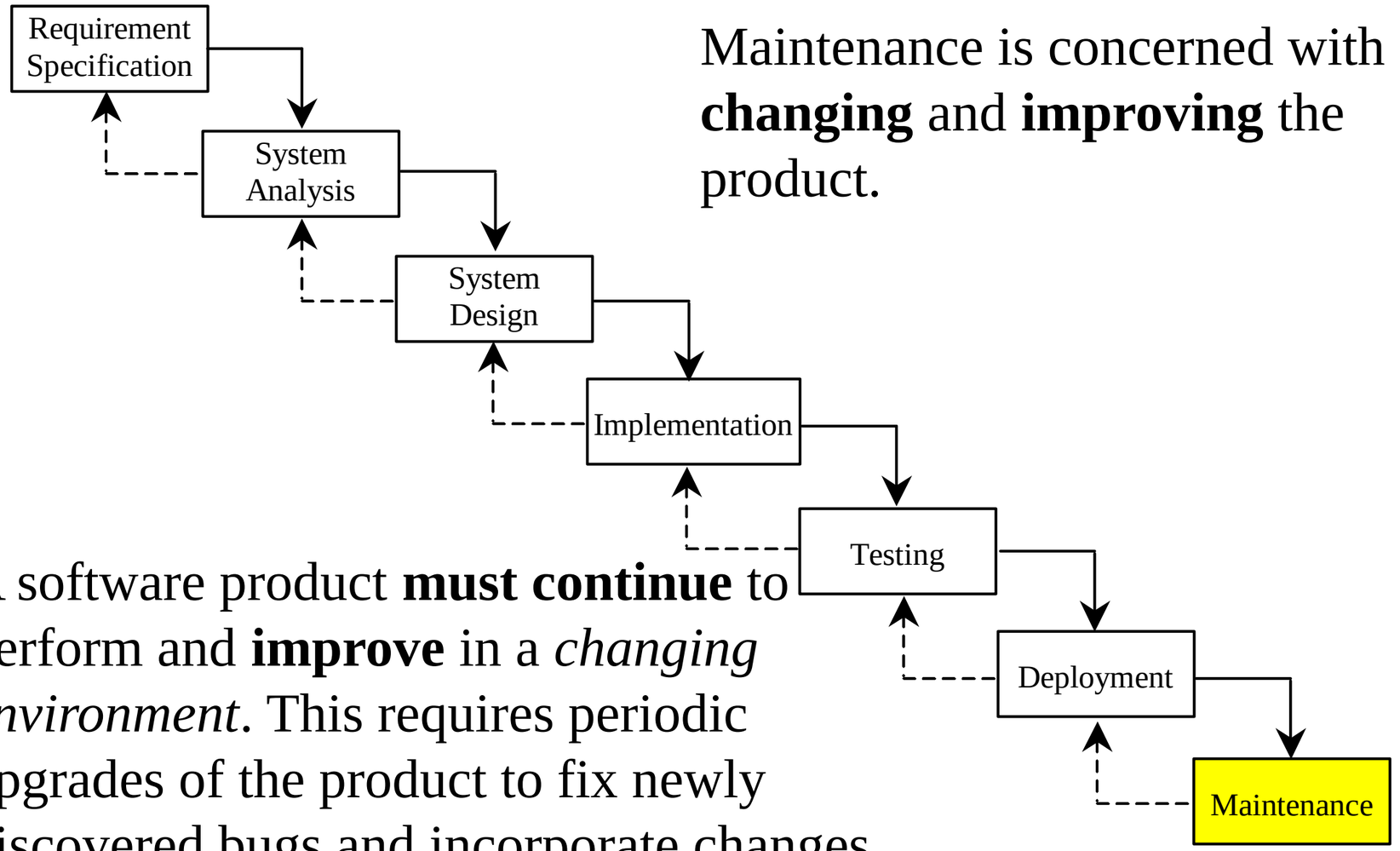
An **independent team** of software engineers **not** involved in implementation of the project usually conducts such testing.

Deployment



Deployment makes the project available for use.

Maintenance



Maintenance is concerned with **changing** and **improving** the product.

A software product **must continue** to perform and **improve** in a *changing environment*. This requires periodic upgrades of the product to fix newly discovered bugs and incorporate changes.

Case Studies…

Problem: Computing Loan Payments

This program lets the user enter the interest rate, number of years, and loan amount, and computes monthly payment and total payment.

$$monthlyPayment = \frac{loanAmount \times monthlyInterestRate}{1 - \frac{1}{(1 + monthlyInterestRate)^{numberOfYears \times 12}}}$$

Stage 1: Requirements Specification

- The program must **satisfy** the following requirements:
  - It must let the user **enter** the *interest rate*, the *loan amount*, and the *number of years* for which payments will be made.
  - It must **compute** and **display** the monthly payment and total payment amounts.

Stage 2: System analysis

- **Input**
  - It must let the user **enter** the *interest rate*, the *loan amount*, and the *number of years* for which payments will be made.
- **Processing**
  - It must **compute** the monthly payment and total payment amounts.
- **Output**
  - It must **display** the monthly payment and total payment amounts.

$$monthlyPayment = \frac{loanAmount \times monthlyInterestRate}{1 - \frac{1}{(1 + monthlyInterestRate)^{numberOfYears \times 12}}}$$

$$totalPayment = monthlyPayment \times numberOfYears \times 12$$

Stage 3: System Design

- **Step 1:** Prompt the user to **enter** the annual interest rate, number of years, and loan amount.
- **Step 2:** The input for the annual interest rate is a number in percent format, such as 4.5%.The program needs to
  - Convert it into a decimal by **dividing** it by **100**.
  - To obtain the **monthly interest rate** from the annual interest rate, **divide it by 12**, since a year has 12 months.
  - So to obtain the monthly interest rate in decimal format, you need to divide the annual interest rate in percentage by **1200**.
  - For example, if the annual interest rate is 4.5%, then the monthly interest **4.5/1200 = 0.00375**.

Stage 3: System Design

- **Step 3:**
  - Compute the monthly payment using the formula given in Stage 2.
- **Step 4:**
  - Compute the total payment, which is the monthly payment multiplied by 12 and multiplied by the number of years.
- **Step 5:**
  - Display the monthly payment and total payment.

Stage 4: Implementation

- **Stage 4:** Implementation
  - Implementation is also known as coding (writing the code).

```
# Enter annual interest rate as a percentage, e.g., 7.25
annualInterestRate = eval(input("Enter annual interest rate, e.g., 7.25: "))
monthlyInterestRate = annualInterestRate / 1200

# Enter number of years
numberOfYears = eval(input("Enter number of years as an integer, e.g., 5: "))

# Enter loan amount
loanAmount = eval(input("Enter loan amount, e.g., 120000.95: "))

# Calculate payment
monthlyPayment = loanAmount * monthlyInterestRate / (1 - 1 / (1 + monthlyInterestRate) ** (numberOfYears
* 12))
totalPayment = monthlyPayment * numberOfYears * 12

# Display results
print("The monthly payment is", int(monthlyPayment * 100) / 100)
print("The total payment is", int(totalPayment * 100) /100)
```

Case Study: Computing Distances

This program prompts the user to enter two points, computes their distance.

```
# Enter the first point with two float values
x1, y1 = eval(input("Enter x1 and y1 for Point 1: "))

# Enter the second point with two float values
x2, y2 = eval(input("Enter x2 and y2 for Point 2: "))

# Compute the distance
distance = ((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2)) **
0.5

print("The distance between the two points is", distance)
```

Comments…

Comments in Python

Anything after a # is ignored by Python

Why comment?

- Describe what is going to happen in a sequence of code
- Document who wrote the code or other ancillary information
- Turn off a line of code - perhaps temporarily

```
# Get the name of the file and open it
name = input('Enter file:')
handle = open(name, 'r')

# Count word frequency
counts = dict()
for line in handle:
    words = line.split()
    for word in words:
        counts[word] = counts.get(word,0) + 1

# Find the most common word
bigcount = None
bigword = None
for word,count in counts.items():
    if bigcount is None or count > bigcount:
        bigword = word
        bigcount = count

# All done
print(bigword, bigcount)
```

Multi-Line Comments

```
'''

This is multiline

comments

'''
```

Exercise

Write a program to prompt the user for hours and rate per hour to compute gross pay.

```
Enter Hours: 35
Enter Rate: 2.75

Pay: 96.25
```

Summary

Type

Reserved words

Variables (mnemonic)

Operators

Operator precedence

Integer Division

Conversion between types

User input

Software Process

Comments (#)